

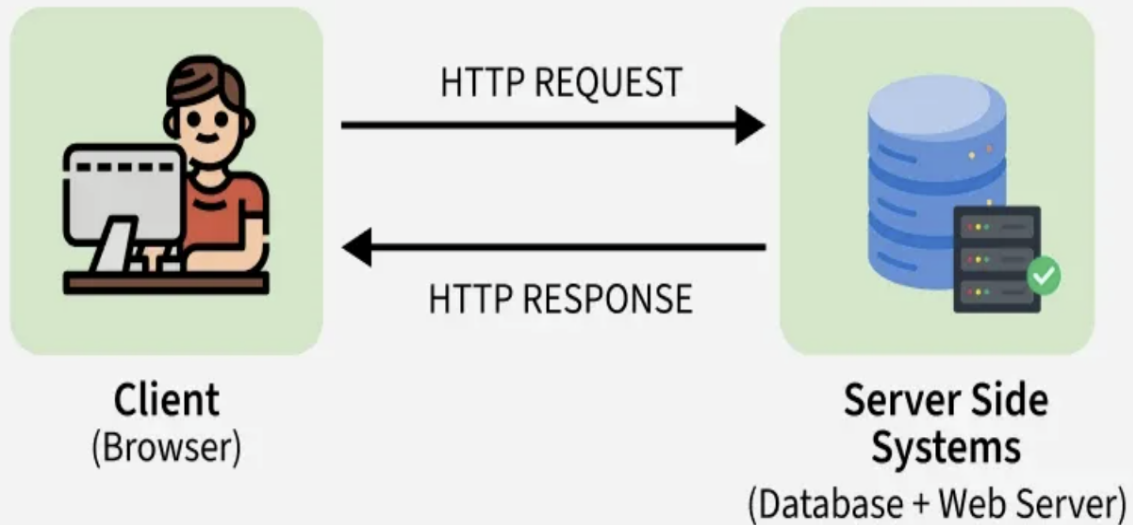
24CSA212 – Full Stack Frameworks

REQUEST AND RESPONSE HANDLING, ERROR HANDLING
AND DEBUGGING TECHNIQUES

HTTP Request and Response Cycle in Express.js

- The HTTP request and response cycle is the backbone of secure communication on the web.
- In Express.js, req (request) and res (response) objects are fundamental to handling HTTP requests and sending back responses.
- They are passed as arguments to route handlers and middleware functions.

Request and Response Cycle



- The client, i.e., the browser, will send an HTTP request to the server-side systems(which include a web server and database).
- For Example- When you visit a website like <https://www.amazon.ac.in/user>, your browser will send an HTTP request to the server asking for the user page.
- The server processes this request—possibly fetching data from a database—and then sends back an HTTP response, such as an HTML page, which the browser displays to the user.

Basics

In Express.js, every route handler (or middleware) receives two main objects:

req (Request) → Represents the incoming HTTP request.

Contains data like URL, query parameters, body, headers, cookies, etc.

res (Response) → Represents the outgoing HTTP response.

Used to send data, set headers, status codes, and end the request.

Request Object

- The Request Object allows you to submit a POST or GET request to a URL.
- Essentially, it provides a way to make a REST API request to another URL.
- You can also send HEAD, PUT, DELETE, OPTIONS, and PATCH requests.

Commonly Used req Properties

Property	Description
1. req.params	Route parameters (e.g., /user/:id)
2. req.query	Query string parameters (e.g., ?name=John)
3. req.body	Request body (requires body-parsing middleware)
4. req.headers	HTTP request headers
5. req.method	HTTP method (GET, POST, etc.)
6. req.url	Full request URL

Response Object

The response object is passed to the second parameter on the request handler, thus we can send to the user a page, an image or download a file.

Commonly Used res Methods

Method	Description
1. res.send()	Sends a response (string, object, buffer, etc.)
2. res.json()	Sends a JSON response
3. res.status()	Sets HTTP status code
4. res.redirect()	Redirects to another URL
5. res.render()	Renders a view template.
6. res.set()	Sets response headers
7. res.end()	Ends the response without data

Methods to Send Request to Server

1. Client Sends a Request

The cycle starts when a clients - such as browser , mobile app or API testing tool(like postman)- sends an HTTP request to the server.

This request includes:

1. HTTP method(e.g GET, PUT, POST, DELETE)
2. URL/EndPoint(e.g, /users, /products/1)
3. Headers(e.g, content-type, authorization)
- 4.Optional Data(like form-data, or JSON in the request body)

2. Express Recieves the Request

Express.js listen for incoming requests on specified routes and HTTPS methods. It matches the request to the appropriate route handler

3. Middleware Processing

Before Reaching the router handler, the request can pass through one or more middleware functions. Middleware can modify the req (request) or perform actions like authentication , logging, or parsing data.

4. Route Handler Executes

The matched route executes its callback function, where you can access request data(req) and prepare a response object using the res object.

5. Server Sends a Response

Using the res object , Express sends the response back to the client. You can send plain text, JSON , HTML or status code.

6. Cycle Completes

Once the response is sent , the cycle ends. The client receives the result, and may act on it or display it to the user.

Status Code

When a server responds to client request in Express.js , it returns more than just a data - it also checks an HTTP Status code.

These codes are 3- digit numbers that indicates the result of the request.

It helps the client to understand whether the request is successful, failed or requires further action.

To use appropriate status code improves communication between client and server, enhances debugging and ensures better API design.

Status Code	Meaning	Example
200 OK	Success	<code>res.status(200).send('Success')</code>
201 Created	Resource Created	<code>res.status(201).json({message: 'User Created'});</code>
204 No Content	Success with no Response Body	<code>res.status(204).send();</code>
400 Bad Request	Client Error	<code>res.status(400).send('Client Error');</code>
401 Unauthorized	Authentication Required	<code>res.status(401).json({error: "Unauthorized"});</code>
403 Forbidden	Access Denied	<code>res.status(403). send('Access Denied');</code>
404 Not Found	Resource Not Found	<code>res.status(404). send('Resource Not Found');</code>
500 Internal Server Error	Internal Server Error	<code>res.status(500). send(' Internal Server Error');</code>

Debugging techniques in Express

1. Use the Built-in debug Module

Express internally uses the debug package for logging.

You can enable it by setting the DEBUG environment variable

```
npm install debug
```

To see all internal logs used in Express, set the DEBUG environment variable to **Express:*** when starting the app –

Example:

Enable only router logs

DEBUG=express : router node app.jsindex.js

Common namespaces:

express:* → All logs

express:router → Router-level logs

express:application → App-level logs

Enable all Express debug logs

DEBUG=express:* node app.js

Example: E:\Dev\hello-world>set DEBUG=express:*

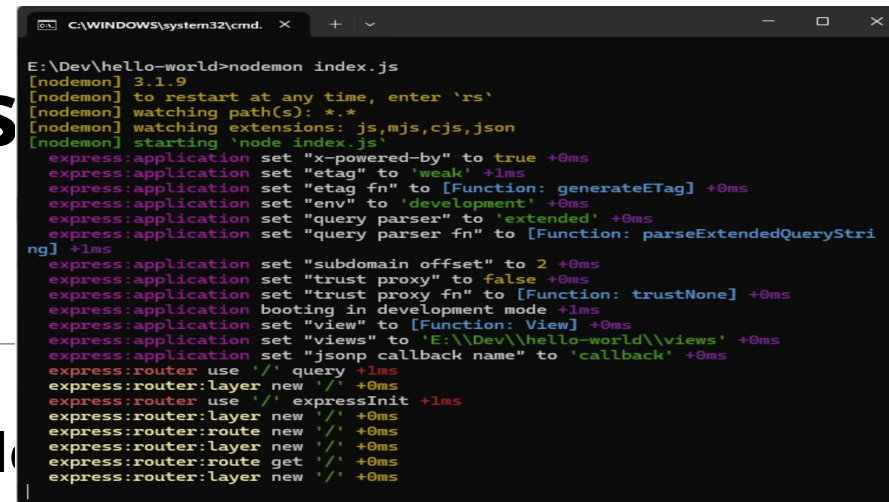
E:\Dev\hello-world>nodemon

The following output will be displayed.

These logs are very helpful when a component of your app is not functioning right. This verbose output might be a little overwhelming. You can also restrict the DEBUG variable to specific area to be logged. For example, if you wish to restrict the logger to application and router, you can use the following code.

DEBUG = express:application, express:router node index.js

Debug is turned off by default and is automatically turned on in production environment.

A screenshot of a Windows command prompt window titled 'C:\WINDOWS\system32\cmd.' The terminal shows the execution of 'nodemon index.js' in the directory 'E:\Dev\hello-world'. The output displays various debug logs from the 'express' package, including messages about restarting, watching paths, and setting application properties like 'x-powered-by', 'etag', 'env', 'query parser', 'subdomain offset', 'trust proxy', 'view', 'views', 'jsonp callback name', 'router', and 'layer'.

Debugging techniques in Express.js

2. Use console.log() Strategically

While basic, console.log() is still useful for quick checks.

Use console.error() for errors.

Use console.table() for structured data.

Remove or comment them out in production.

3. Use Node.js Inspector for Step-by-Step Debugging

Node.js has a built-in debugger.

```
node --inspect app.js
```

Then open chrome://inspect in Chrome and attach to the process.

You can set breakpoints, step through code, and inspect variables.

Debugging techniques in Express.js

4. Use VS Code Debugger

Create .vscode/launch.json:

```
{  
  "version": "0.2.0",  
  "configurations": [  
    {  
      "type": "node",  
      "request": "launch",  
      "name": "Debug Express App",  
      "program": "${workspaceFolder}/app.js"  
    }  
  ]  
}
```

Press **F5** to start debugging with breakpoints.

Debugging techniques in Express.js

5. Use Middleware for Request/Response Logging

Install morgan for HTTP request logging:

```
npm install morgan
```

Example:

```
const express = require('express');  
const morgan = require('morgan');  
const app = express();
```

```
app.use(morgan('dev')); // Logs method, URL, status, response time
```

```
app.get('/', (req, res) => res.send('Hello World'));  
app.listen(3000, () => console.log('Server running on port 3000'));
```

Debugging techniques in Express.js

6. Handle Errors with Custom Middleware

Express error-handling middleware helps catch and log errors.

Example:

```
app.use((err, req, res, next) => {  
  console.error(err.stack);  
  res.status(500).json({ error: 'Something went wrong!' });  
});
```

7. Use nodemon for Auto-Restart

```
npm install --save-dev nodemon  
nodemon app.js
```

This restarts the server automatically on file changes, speeding up debugging.